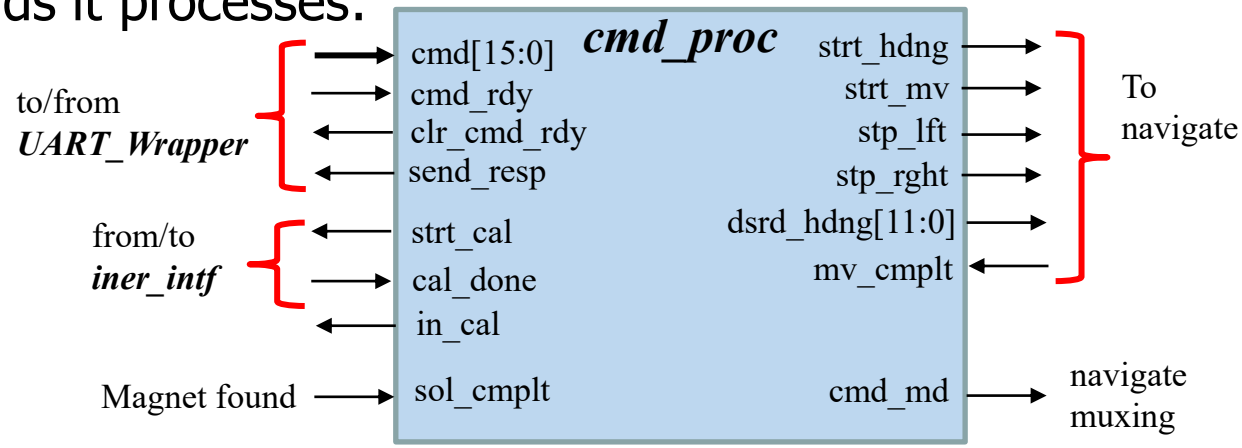
Exercise 19: cmd_proc & piezo_drv

- This is a group exercise – one person submits all files & make sure to include rest of the group in the submission

- Recommended split
  - 2 people on **cmd_proc** implementation & test
  - 2 people on **piezo_drv** implementation & test

Exercise 19: cmd_proc

- The command processor (cmd_proc) unit's main job is to process incoming commands from Bluetooth (UART_wrapper)
- There are 4 commands it processes:
  - Calibrate gyro
  - Heading
  - Movement
  - Solve maze

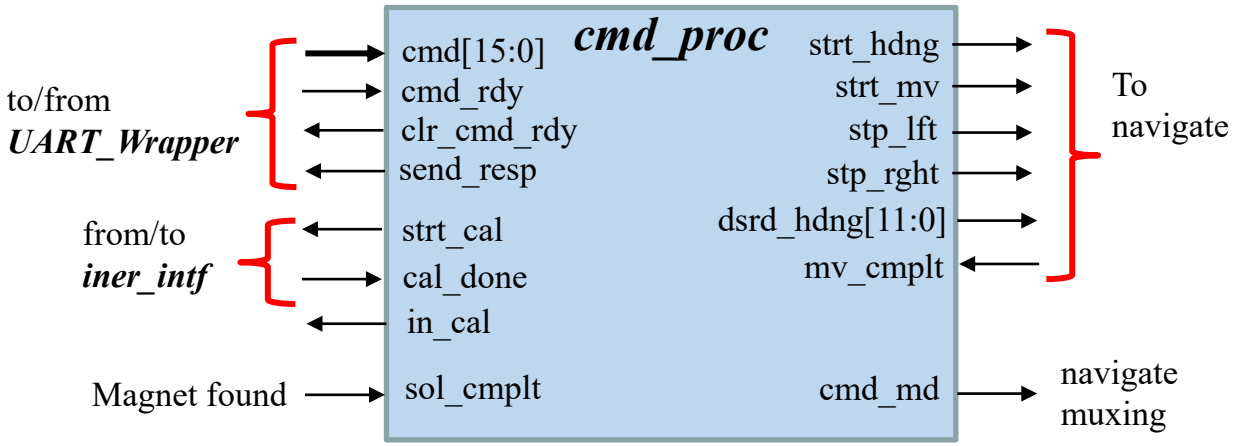


- When a new **cmd**[15:0] is **rdy**, the **cmd_proc** unit will dispatch from its IDLE state to a state that "execute" the command. Assert **clr_cmd_rdy** in the process.
- When it finishes a cmd, assert **send_resp** (UART_wrapper will send 0xA5)

Calibrate gyro cmd:

- **strt_cal** will be asserted for 1 clk
- The unit will wait for **cal_done** to be asserted
- **in_cal** should be asserted during this time
- When **cal_done** is asserted, the unit should **send_resp** and return to IDLE

Exercise 19: cmd_proc



Heading cmd:

- Heading direction should be registered into **dsrd_hdng**
- **strt_hdng** should be asserted for 1 clk
- When the MazeRunner is at the desired heading, **mv_cmplt** will be asserted and the unit will **send_resp** and return to IDLE

Movement cmd:

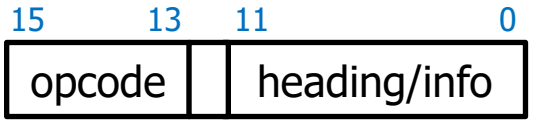
- **strt_mv** should be asserted for 1 clk
- Register if the movement should stop at a left or right opening in the maze (**stp_lft**, **stp_rght)** based on **cmd**[1:0]
- When ***navigate*** is done executing the move, **mv_cmplt** will be asserted and the unit will **send_resp** and return to IDLE

Solve cmd:

- Simply deassert **cmd_md** (should have been high otherwise) and wait for **sol_cmplt** to be asserted (sol_cmplt asserts when the hall sensor detects a magnet)
- The "intelligence" of solving the maze is in a unit called ***maze_solve***

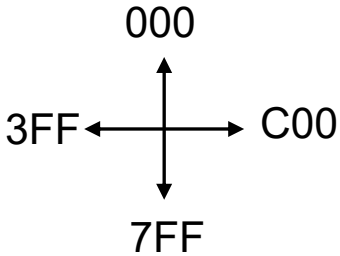
Exercise 19: cmd_proc

- The commands have the following format:



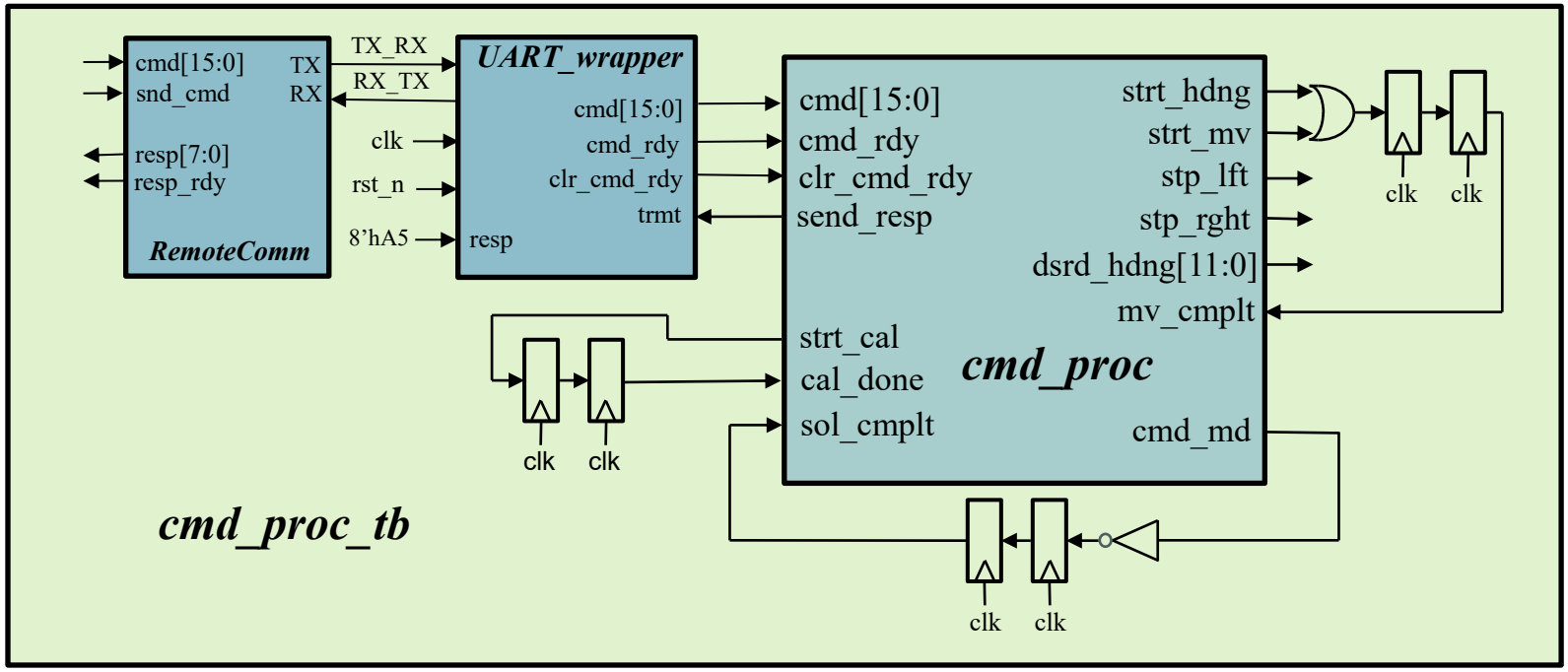
cmd[15:13]	Description
3'b000	Calibrate command (kick off **strt_cal** and wait for **cal_done**). **send_resp**
3'b001	Heading command. **cmd**[11:0] represent the desired heading
3'b010	Move command. **cmd**[1] if set implies movement should stop at a left opening. **cmd**[0] if set implies movement should stop at a right opening.
3'b011	Start solve command: Deassert **cmd_md** and wait for **sol_cmplt**. **cmd**[0] determines affinity. 1→ left affinity, 0→ right affinity.
3'b1xx	Reserved for future expansion.

cmd[11:0] → Heading	Direction
12'h000	North
12'h3FF	West
12'h7FF	South
12'hC00	East



*assume heading is north after calibration

Exercise 19: cmd_proc Testing



- Since ***cmd_proc*** is so central to the design, it has an extensive interface. For blocks like this it can be good to instantiate in a testbench with some context.
- In this recommended context you can apply a 16-bit command to **cmd[15:0]** into ***RemoteComm*** and then assert **snd_cmd** for one clock. This will send the command and you can check for its proper operation.
- **NOTE:** the use of flops to provide **cmplt/done** signals a few clks after their respective **strt** signals.

Exercise 19: cmd_proc Testing

- There is a lot to test with ***cmd_proc***, and some of it might be better tested at the actual "fullchip" level when a physics model of the mazeRunner and maze are available. However, the following are tests you should consider performing at this level:

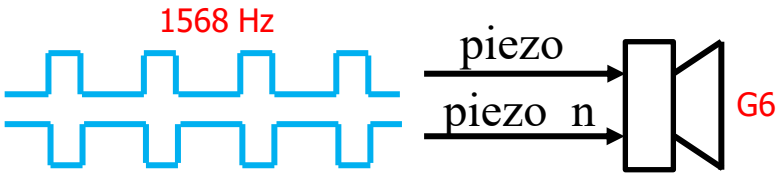
Sequence	Description
1	Initialize all inputs to known values and assert/deassert reset
2	Send Calibrate command (0x0000). Wait for **cal_done** or timeout if it does not occur Wait for **resp_rdy** or timeout if it does not occur
3	Send heading command and look for **strt_hdng** at time of **cmd_rdy**. One clock later check that **dsrd_hdng** is heading direction you sent. Check that it sends a 0xA5 response when **cal_cmplt** occurs.
4	Send move command and look for **strt_mv** at time of **cmd_rdy**. One clock later check that either **stp_lft** or **stp_rght** (depending) is asserted.
5	Send solve command and look for **cmd_md** to be low one clock after **cmd_rdy**.

Exercise 19: piezo_drv

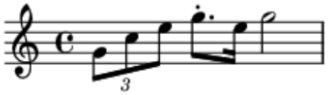
Piezo Bender

- Certain materials (e.g., quartz crystal) produce voltage when they are mechanically stressed, and their shape deform when voltage is applied → piezoelectric effect
- Piezo bender – a "speaker" that uses piezoelectric effect to generate sound
  - A metal layer sandwiched between 2 piezoelectric ceramic layers
  - When voltage applied, one contracts and one expands

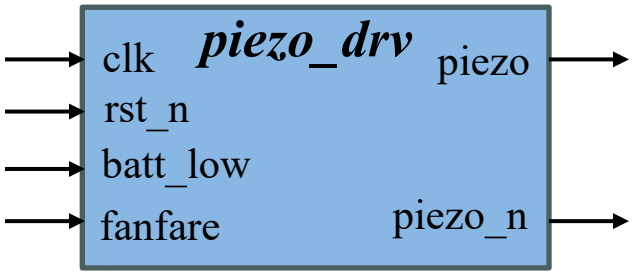
- We can drive the piezo bender using GPIO pins on the FPGA
  - Use square wave of the frequency we want to generate a tone for
  - Duty cycle is not so important (anything from 20% to 80% will do)
  - We will drive differentially for increased amplitude



Exercise 19: piezo_drv

- There are 2 sound sequences the MazeRunner will make:
  - "Charge!" fanfare when the magnet is found 
  - The notes G6, C7, E7 continuously when the battery is low

- Battery low has priority over magnet found
- Once "Charge!" is started it is allowed to complete (no need to preempt "charge!" if battery becomes low)

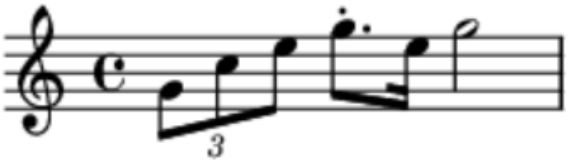


Signal	Dir	Description
clk,rst_n	In	50MHz clk & asynch reset
batt_low	In	Asserted if battery voltage is low (has priority over fanfare)
fanfare	In	Asserted if magnet has been found
piezo,piezo_n	out	Drive the piezo buzzer with desired note frequency for desired duration

Exercise 19: piezo_drv

- When the MazeRunner encounters the magnet, it will play "Charge!" Fanfare

Note	**Freq**	**Duration**
G6 (G in octave 6)	1568	2^{23} clocks
C7 (C in octave 7)	2093	2^{23} clocks
E7	2637	2^{23} clocks
G7	3136	$2^{23} + 2^{22}$ clocks
E7	2637	2^{22} clocks
G7	3136	2^{24} clocks



NOTE: The battery low sound *(G6,C7,E7 played for 2^{23} clocks each)* is comprised of 3-notes that were already part of "Charge!"

- You will need 2 counters in the design:
  - A note frequency counter
  - A note duration counter

Exercise 19: piezo_drv

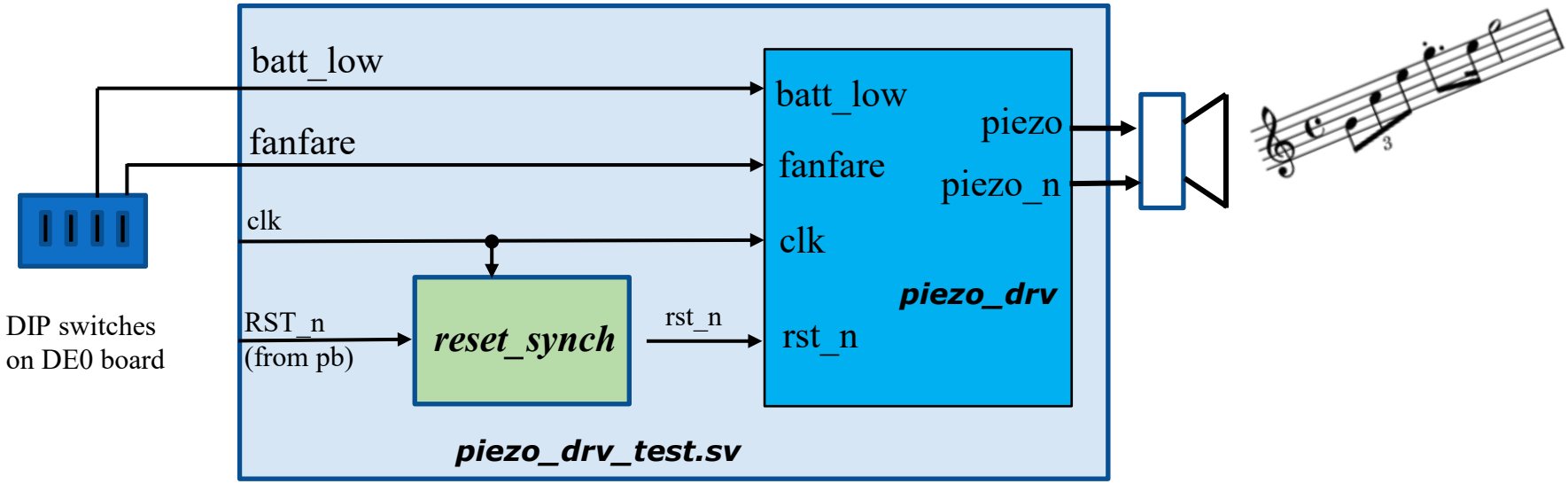
- The duration of the notes (2^{23} clock cycles) is quite long for simulation *(especially when we get to fullchip simulations where **piezo_drv** will be one of many blocks being simulated)*
- We will use FAST_SIM parameter to speed up simulation

- When FAST_SIM=1, duration of the notes should be 1/16 their normal length
- **HINT**: Use a generate if statement to create an amount by which you increment your duration counter (i.e., increment by 1 or by 16 depending on FAST_SIM)

- Create a simple testbench (**piezo_drv_tb.sv**) that simply instantiates DUT, applies clk and reset, and asserts **fanfare** for 1 clk.
- **Piezo** and **piezo_n** should not be toggling before **fanfare** is asserted or after the "tune" has completed.
- Make sure you are testing **batt_low** as well.

- Once your testbench is passing (visual inspection of piezo/piezo_n is enough), move on to testing with the DE0-nano

Exercise 19: piezo_drv



- Create **piezo_drv_test.sv** to map your **piezo_drv.sv** to the DE0 and test in "real life". Don't forget to set FAST_SIM=0 since we are testing it on real HW now.

Signal	**Dir**	**Description**
clk	in	50MHz clock
RST_n	in	Unsynchronized input from push button
Batt_low	In	Indicates battery is low
fanfare	in	Magnet has been found, "Charge!" fanfare should be played
piezo/piezo_n	out	Differential drive of piezo bender

Exercise 19: piezo_drv

- There are Quartus project file and settings file available for download (**piezo_drv_test.qpf, piezo_drv_test.qsf)**

- Open the .qpf and ensure you add all necessary files to the project
- Ensure the project builds in Quartus with no errors

Demo

- Call anyone on the teaching staff over to demo on a DE0 test platform
  - Bonus credit: +0.5 if your group demos a correct implementation by Friday